

A Course End Project Report on

Database-Driven Airline Reservation and Management System

A9512- DATABASE MANAGEMENT SYSTEM LABORATORY

Submitted by

K.Sai Varshitha	25881A0547
M.Sathvika Reddy	25881A0552
K.Vaishnavi	25881A0558

Course Facilitator

Mrs. Shaik Shahanaz
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

26 Years of Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled "Database-Driven Airline Reservation and Management System" is carried out by "K.Sai Varshitha","M.Sathvika Reddy"and "K.Vaishnavi" with Roll Numbers "25881A0547","25881A0552" and "25881A0558" towards A9512– DataBase Management System Laboratory course in partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering during the Academic year 2026-27.

Course Facilitator
Mrs. Shaik Shahanaz
Assistant Professor, CSE

Academic Head
Dr. R. Karthikeyan
Professor, CSE

Head of the Department
Prof. Y. Vijayalata
Department of CSE

Contents

1 Introduction 5

2 Problem Analysis 5
2.1 Problem Statement 5
2.2 Users and Assumptions 5
2.3 Functional Requirements 6

3 ER Model 6
3.1 Entities and Attributes 6
3.2 Relationships 7
3.3 Cardinality and Participation 7
3.4 ER Diagram 7

4 Relational Model 9
4.1 Relational Schema 9
4.2 Primary Keys and Foreign Keys 9

5 Normalization 10
5.1 Functional Dependencies 10
5.2 First Normal Form (1NF) 10
5.3 Second Normal Form (2NF) 11
5.4 Third Normal Form (3NF) 11

6 SQL DDL Commands 11
6.1 Table Creation 12
6.2 Constraints 14
6.3 ALTER Operation 14
6.4 DROP Operation 14

7 SQL DML Commands 14
7.1 INSERT 15
7.2 UPDATE 15
7.3 DELETE 15

8 SQL Queries 16
8.1 Basic SELECT Queries 16
8.2 Joins 16
8.3 Aggregate Functions 17
8.4 GROUP BY and HAVING 17
8.5 Subqueries 18
8.6 Views 18

9 Transactions and Concurrency 19
9.1 Transaction Management 19
9.2 COMMIT, ROLLBACK and SAVEPOINT 19
9.3 Concurrency Control 20

10 PL/SQL Implementation 22

10.1 Functions 23
10.2 Stored Procedures 24
10.3 Cursors 25
10.4 Triggers 26

11 File Organization and Indexing 28
11.1 File Organization 29
11.2 ISAM 29
11.3 B+ Tree Indexing 30

12 Testing 30
12.1 Test Cases 31
12.2 Normal and Boundary Cases 31
12.3 Invalid Input Testing 31
12.4 Transaction and Concurrency Testing 32

13 Reports 33
13.1 Booking Report 33
13.2 Cancellation Report 33
13.3 Revenue Report 33

14 Results and Discussion 34

15 Conclusion 35

16 SDG Mapping 35

17 Future Scope 36

18 References 36

Abstract

The Airline Reservation System is a database-based application developed to simplify and efficiently manage the process of booking airline tickets. It maintains details of airports, aircraft, flights, flight schedules, seats, passengers, and bookings in a structured manner. The system allows users to search for flights between two cities, check seat availability on a particular date, select seats, and reserve tickets.

The system also manages booking and cancellation details, calculates refunds, and supports the generation of useful reports such as booking, cancellation, and revenue reports. By using a centralized database, it helps reduce problems such as double booking of seats, incorrect seat availability, and difficulty in maintaining records. The project demonstrates important DBMS concepts such as ER modeling, normalization, SQL queries, constraints, transactions, concurrency control, PL/SQL functions, procedures, cursors, triggers, file organization, and indexing to provide a reliable and efficient reservation system.

Keywords: Airline Reservation System, SQL, Oracle Database, DBMS, Relational Database, Airport, Aircraft, Flight, Schedule, Seat, Passenger, Booking, Primary Key, Foreign Key, Joins, Aggregate Functions, Subqueries, Views, Transactions, Triggers, B+ Tree

1 Introduction

The Airline Reservation System is a database-based application developed to simplify and efficiently manage the process of booking airline tickets. It maintains details of airports, aircraft, flights, flight schedules, seats, passengers, and bookings in a structured manner. The system allows users to search for flights and available dates, check seat availability, select seats, and reserve tickets.

The system also manages booking and cancellation details and supports the generation of useful reports such as booking and revenue reports. By using a centralized database, it helps reduce problems such as double booking, incorrect seat information, and difficulty in maintaining records. The project demonstrates important DBMS concepts such as SQL queries, normalization, constraints, transactions, concurrency control, PL/SQL functions and procedures, cursors, triggers, and indexing to provide a reliable and efficient reservation system.

The project is implemented in Oracle SQL and PL/SQL. All SQL scripts used in this report are also supplied in the `sql/` folder accompanying this report so that every result can be reproduced.

2 Problem Analysis

2.1 Problem Statement

Airlines operate many flights every day, and each flight runs on several dates with a limited number of seats. When flight, seat, passenger, and ticket information is handled manually or in unrelated files, it leads to double booking of the same seat, wrong seat availability, lost cancellation records, and delays in preparing revenue reports. The proposed Airline Reservation System provides a centralized relational database that stores all of these details consistently and ensures that every seat on every flight date is sold at most once.

2.2 Users and Assumptions

The main users of the system are passengers and administrators. Passengers can search for flights, check seat availability, select seats, and make or cancel bookings. Administrators manage airports, aircraft, flights, schedules, seats, passengers, and booking records, and generate reports.

The following assumptions are made:

- A *flight* (for example AI101) is a route operated by one aircraft between a source and a destination airport; a *schedule* is one dated operation of that flight.
- Every booking belongs to one registered passenger and reserves exactly one seat on one schedule; a passenger who needs several seats makes several bookings.
- A seat of an aircraft may be reused on different dates but can have only one confirmed booking per schedule.
- A cancelled booking is never deleted; its status is changed to *Cancelled* so that history is retained, and the seat becomes available again.
- Fares are stored in Indian Rupees; a cancelled booking is refunded 80% of the fare.

2.3 Functional Requirements

- **FR1:** Maintain airport, aircraft, flight, schedule, seat, passenger, and booking data.
- **FR2:** Search flights by source, destination, and travel date.
- **FR3:** Display the available seats of a scheduled flight.
- **FR4:** Book a seat for a registered passenger and calculate the fare.
- **FR5:** Prevent duplicate reservation of a seat on the same flight date, including when two users book at the same time.
- **FR6:** Cancel a booking, restore the seat, and calculate the refund.
- **FR7:** Automatically update seat inventory and keep an audit history of every booking change.
- **FR8:** Generate booking, cancellation, and revenue reports.

3 ER Model

The Entity-Relationship (ER) model represents the structure of the Airline Reservation System and the relationships between its entities. The main entities are Airport, Aircraft, Flight, Schedule, Seat, Passenger, and Booking. Each entity is described by attributes and identified by a primary key.

3.1 Entities and Attributes

Entity	**Attributes**	**Foreign Keys**
Airport	Airport_ID, Airport_Name, City	–
Aircraft	Aircraft_ID, Model, Capacity	–
Flight	Flight_ID, Flight_Number, Dep_Time, Arr_Time, Base_Fare	Aircraft_ID, Source_Airport, Dest_Airport
Schedule	Schedule_ID, Flight_Date, Available_Seats	Flight_ID
Seat	Seat_ID, Seat_Number, Seat_Class	Aircraft_ID
Passenger	Passenger_ID, Passenger_Name, Email, Phone	–
Booking	Booking_ID, Booking_Date, Fare, Status	Passenger_ID, Schedule_ID, Seat_ID

Table 1: Entities, attributes, primary keys and foreign keys

Airport stores the airports served. Aircraft stores the aircraft models and their seating capacity. Flight stores a route with its timings and base fare. Schedule stores each dated operation of a flight together with the number of seats still available. Seat maintains the seat numbers and classes of every aircraft. Passenger stores customer details, and Booking records the tickets reserved by passengers.

3.2 Relationships

An airport can be the source of many flights and the destination of many flights, while each flight has exactly one source and one destination airport. An aircraft operates many flights and contains many seats. A flight has many schedules (one per travel date). A passenger makes many bookings. A schedule includes many bookings, and a seat can be assigned to many bookings (on different dates), while each booking is for one passenger, one schedule, and one seat.

3.3 Cardinality and Participation

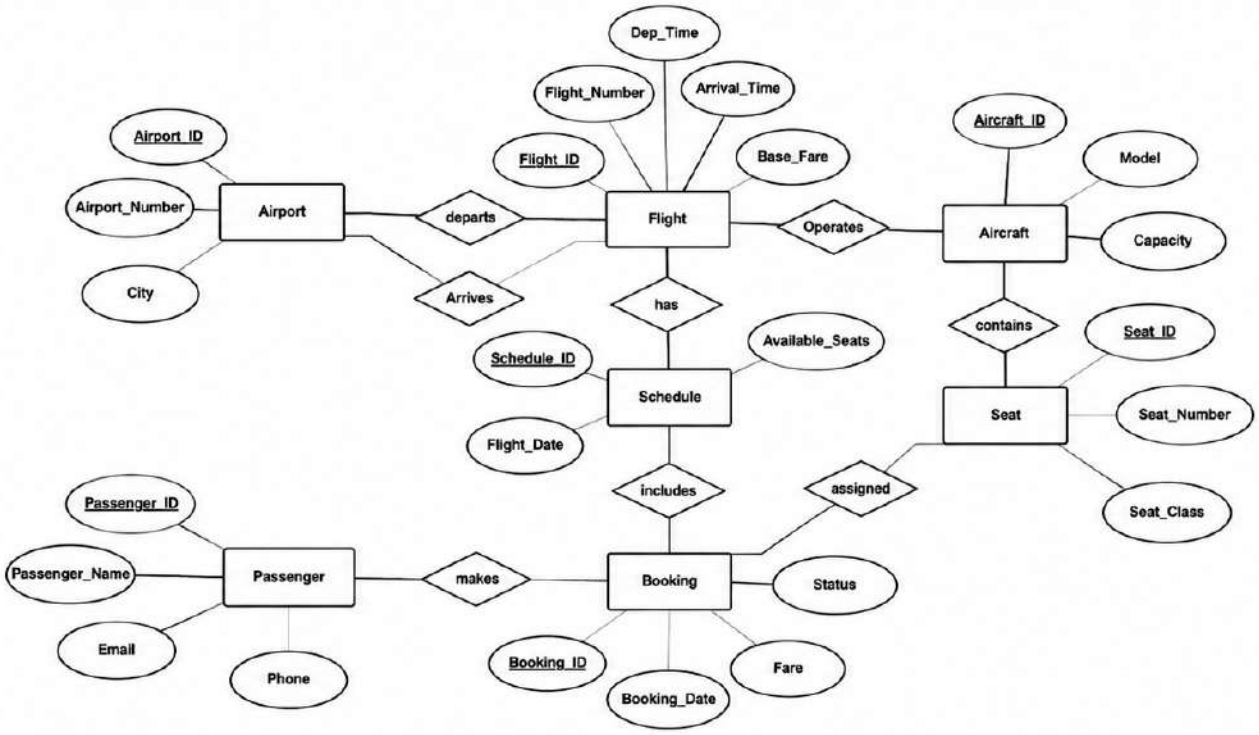
Relationship	**Entities**	**Cardinality**	**Participation**
departs	Airport – Flight	1 : N	Flight total, Airport partial
arrives	Airport – Flight	1 : N	Flight total, Airport partial
operates	Aircraft – Flight	1 : N	Flight total, Aircraft partial
has	Flight – Schedule	1 : N	Schedule total, Flight partial
contains	Aircraft – Seat	1 : N	Seat total, Aircraft partial
makes	Passenger – Booking	1 : N	Booking total, Passenger partial
includes	Schedule – Booking	1 : N	Booking total, Schedule partial
assigned	Seat – Booking	1 : N	Booking total, Seat partial

Table 2: Cardinality and participation constraints

All relationships are one-to-many. Booking participates totally in its relationships with Passenger, Schedule, and Seat because every booking must belong to a passenger, a schedule, and a seat. Entities such as Airport, Aircraft, and Passenger can exist independently before any flight or booking is created.

3.4 ER Diagram

The ER diagram on the next page shows the entities, attributes, primary keys (underlined), relationships, cardinalities, and participation (double lines mean total participation). Foreign keys are not drawn as attributes because they are represented by the relationships; they appear when the diagram is converted to tables in Section 4.


Dep_Time
Flight_Number
Arrival_Time
Airport_ID
Flight_ID
Base_Fare
Aircraft_ID
Model
Airport_Number
Airport
departs
Flight
Operates
Aircraft
Capacity
City
Arrives
has
contains
Seat_ID
Schedule_ID
Available_Seats
Schedule
Seat
Seat_Number
Flight_Date
includes
assigned
Seat_Class
Passenger_ID
Passenger_Name
Passenger
makes
Booking
Status
Email
Phone
Booking_ID
Fare
Booking_Date

4 Relational Model

The relational model represents the data of the Airline Reservation System in the form of tables. The ER model is converted using the standard mapping rules: every entity becomes a table whose key is its primary key, and for every one-to-many relationship the primary key of the "one" side is placed as a foreign key in the table on the "many" side.

4.1 Relational Schema

The relational tables used in the system are Airport, Aircraft, Flight, Schedule, Seat, Passenger, and Booking, together with a supporting Booking_Audit table used by the audit trigger. Primary keys are underlined and foreign keys are marked FK.

- **Airport**(*Airport_ID*, Airport_Name, City)
- **Aircraft**(*Aircraft_ID*, Model, Capacity)
- **Flight**(*Flight_ID*, Flight_Number, Aircraft_ID^{FK}, Source_Airport^{FK}, Dest_Airport^{FK}, Dep_Time, Arr_Time, Base_Fare)
- **Schedule**(*Schedule_ID*, Flight_ID^{FK}, Flight_Date, Available_Seats)
- **Seat**(*Seat_ID*, Aircraft_ID^{FK}, Seat_Number, Seat_Class)
- **Passenger**(*Passenger_ID*, Passenger_Name, Email, Phone)
- **Booking**(*Booking_ID*, Passenger_ID^{FK}, Schedule_ID^{FK}, Seat_ID^{FK}, Booking_Date, Fare, Status)
- **Booking_Audit**(*Audit_ID*, Booking_ID, Action_Type, Old_Status, New_Status, Changed_By, Changed_On)

4.2 Primary Keys and Foreign Keys

Each table has a primary key that uniquely identifies every record. Foreign keys connect related tables.

Table	Foreign key	References
Flight	Aircraft_ID	Aircraft(Aircraft_ID)
Flight	Source_Airport	Airport(Airport_ID)
Flight	Dest_Airport	Airport(Airport_ID)
Schedule	Flight_ID	Flight(Flight_ID)
Seat	Aircraft_ID	Aircraft(Aircraft_ID)
Booking	Passenger_ID	Passenger(Passenger_ID)
Booking	Schedule_ID	Schedule(Schedule_ID)
Booking	Seat_ID	Seat(Seat_ID)

Table 3: Foreign key constraints

Besides the primary keys, the candidate keys Flight_Number, Email, Phone, and (Aircraft_ID, Seat_Number) are declared UNIQUE.

5 Normalization

Normalization is the process of organizing data in a database to reduce redundancy and avoid insertion, deletion, and update anomalies. The Airline Reservation System is normalized up to Third Normal Form (3NF) and every determinant is also a candidate key, so the relations are in BCNF as well.

5.1 Functional Dependencies

The following functional dependencies are identified:

- Airport_ID → Airport_Name, City
- Aircraft_ID → Model, Capacity
- Flight_ID → Flight_Number, Aircraft_ID, Source_Airport, Dest_Airport, Dep_-Time, Arr_Time, Base_Fare
- Flight_Number → Flight_ID (candidate key)
- Schedule_ID → Flight_ID, Flight_Date, Available_Seats
- Seat_ID → Aircraft_ID, Seat_Number, Seat_Class
- (Aircraft_ID, Seat_Number) → Seat_ID, Seat_Class (candidate key)
- Passenger_ID → Passenger_Name, Email, Phone
- Email → Passenger_ID; Phone → Passenger_ID (candidate keys)
- Booking_ID → Passenger_ID, Schedule_ID, Seat_ID, Booking_Date, Fare, Status

5.2 First Normal Form (1NF)

A relation is in First Normal Form when all attributes contain atomic values and there are no repeating groups. Suppose bookings were first recorded in a single sheet where one row holds several seats and a passenger has several phone numbers:

Booking ID	Passenger	Phone	Flight	Seats
B1	Ananya	9876500001, 9876500011	AI101	1A, 1B
B2	Rohit	9876500002	AI101	12C

Table 4: Unnormalized booking sheet (multi-valued attributes)

To reach 1NF, each field is made atomic: the passenger is stored once in the Passenger relation and every seat is stored in a separate Booking row, so one booking row stores exactly one seat.

Booking ID	Passenger ID	Schedule ID	Seat ID	Booking Date	Fare	Status
701	601	301	501	20-10-2026	9500	Confirmed
702	602	301	502	21-10-2026	5500	Confirmed
703	603	302	504	22-10-2026	4200	Cancelled

Table 5: Booking relation in 1NF

5.3 Second Normal Form (2NF)

A relation is in 2NF when it is in 1NF and every non-key attribute depends on the *whole* primary key. Consider a combined ticket relation with the composite key (Booking_ID, Seat_ID):

Ticket(Booking_ID, Seat_ID, Passenger_Name, Fare, Seat_Number, Seat_Class)

Here Booking_ID → Passenger_Name, Fare and Seat_ID → Seat_Number, Seat_Class are *partial dependencies*, because the attributes depend on only a part of the key. They are removed by placing booking details in Booking and seat details in Seat, so that Passenger_Name depends only on Passenger_ID in Passenger, and Seat_Number and Seat_Class depend only on Seat_ID in Seat. In the final design every relation has a single-attribute primary key, so no partial dependency can exist.

5.4 Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and has no transitive dependency between non-key attributes. Consider a flight relation that also stores aircraft and airport details:

Flight_Full(Flight_ID, Aircraft_ID, Model, Capacity, Source_Airport, Source_City)

Here Flight_ID → Aircraft_ID and Aircraft_ID → Model, Capacity, so Model and Capacity depend transitively on Flight_ID; likewise Flight_ID → Source_Airport → Source_City. Repeating the model and city for every flight would cause update anomalies. They are removed by storing only Aircraft_ID and the airport IDs in Flight, and keeping Model, Capacity, and City in the Aircraft and Airport relations. Thus the final relations are:

- Airport(*Airport_ID*, Airport_Name, City)
- Aircraft(*Aircraft_ID*, Model, Capacity)
- Flight(*Flight_ID*, Flight_Number, Aircraft_ID, Source_Airport, Dest_Airport, Dep_Time, Arr_Time, Base_Fare)
- Schedule(*Schedule_ID*, Flight_ID, Flight_Date, Available_Seats)
- Seat(*Seat_ID*, Aircraft_ID, Seat_Number, Seat_Class)
- Passenger(*Passenger_ID*, Passenger_Name, Email, Phone)
- Booking(*Booking_ID*, Passenger_ID, Schedule_ID, Seat_ID, Booking_Date, Fare, Status)

The database is therefore normalized up to 3NF, which reduces data redundancy and improves consistency and maintainability.

6 SQL DDL Commands

Data Definition Language (DDL) commands create and modify the structure of database tables. In the Airline Reservation System, DDL commands create the tables and define constraints such as primary keys, foreign keys, UNIQUE, NOT NULL, and CHECK.

6.1 Table Creation

The following statements create the tables of the system. The complete script is available in `sql/01_ddl.sql`.

```
CREATE TABLE Airport (
    Airport_ID   INT PRIMARY KEY,
    Airport_Name VARCHAR2(100) NOT NULL,
    City         VARCHAR2(50) NOT NULL
);

CREATE TABLE Aircraft (
    Aircraft_ID INT PRIMARY KEY,
    Model       VARCHAR2(50) NOT NULL,
    Capacity    INT CHECK (Capacity > 0)
);

CREATE TABLE Flight (
    Flight_ID      INT PRIMARY KEY,
    Flight_Number  VARCHAR2(10) NOT NULL UNIQUE,
    Aircraft_ID    INT,
    Source_Airport INT,
    Dest_Airport   INT,
    Dep_Time       VARCHAR2(10),
    Arr_Time       VARCHAR2(10),
    Base_Fare      NUMBER(10,2) CHECK (Base_Fare > 0),
    FOREIGN KEY (Aircraft_ID)    REFERENCES Aircraft(Aircraft_ID),
    FOREIGN KEY (Source_Airport) REFERENCES Airport(Airport_ID),
    FOREIGN KEY (Dest_Airport)   REFERENCES Airport(Airport_ID),
    CHECK (Source_Airport <> Dest_Airport)
);

CREATE TABLE Schedule (
    Schedule_ID     INT PRIMARY KEY,
    Flight_ID       INT,
    Flight_Date     DATE NOT NULL,
    Available_Seats INT CHECK (Available_Seats >= 0),
    FOREIGN KEY (Flight_ID) REFERENCES Flight(Flight_ID)
);

CREATE TABLE Seat (
    Seat_ID     INT PRIMARY KEY,
    Aircraft_ID INT,
    Seat_Number VARCHAR2(5) NOT NULL,
    Seat_Class  VARCHAR2(20)
                CHECK (Seat_Class IN ('Economy','Business','First')),
    FOREIGN KEY (Aircraft_ID) REFERENCES Aircraft(Aircraft_ID),
    UNIQUE (Aircraft_ID, Seat_Number)
);

CREATE TABLE Passenger (
    Passenger_ID   INT PRIMARY KEY,
    Passenger_Name VARCHAR2(100) NOT NULL,
    Email          VARCHAR2(100) UNIQUE,
    Phone          VARCHAR2(15) UNIQUE
);

CREATE TABLE Booking (
```

```
    Booking_ID   INT PRIMARY KEY,
    Passenger_ID INT,
    Schedule_ID  INT,
    Seat_ID      INT,
    Booking_Date DATE,
    Fare         NUMBER(10,2) CHECK (Fare > 0),
    Status       VARCHAR2(20)
                 CHECK (Status IN ('Confirmed','Cancelled')),
    FOREIGN KEY (Passenger_ID) REFERENCES Passenger(Passenger_ID),
    FOREIGN KEY (Schedule_ID)  REFERENCES Schedule(Schedule_ID),
    FOREIGN KEY (Seat_ID)      REFERENCES Seat(Seat_ID)
);

-- A seat can be held by only ONE confirmed booking per schedule
CREATE UNIQUE INDEX Uq_Active_Seat ON Booking (
    CASE WHEN Status = 'Confirmed' THEN Schedule_ID END,
    CASE WHEN Status = 'Confirmed' THEN Seat_ID END
);

-- Supporting table used by the audit trigger
CREATE TABLE Booking_Audit (
    Audit_ID    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Booking_ID  INT,
    Action_Type VARCHAR2(10),
    Old_Status  VARCHAR2(20),
    New_Status  VARCHAR2(20),
    Changed_By  VARCHAR2(30) DEFAULT USER,
    Changed_On  DATE DEFAULT SYSDATE
);
```

```
SQL> @01_ddl.sql

Table created.   (Airport)
Table created.   (Aircraft)
Table created.   (Flight)
Table created.   (Schedule)
Table created.   (Seat)
Table created.   (Passenger)
Table created.   (Booking)

Index created.   (Uq_Active_Seat)

Table created.   (Booking_Audit)
```

Figure 2: Output of Table Creation

6.2 Constraints

Constraint	Where used
PRIMARY KEY	Airport_ID, Aircraft_ID, Flight_ID, Schedule_ID, Seat_ID, Passenger_ID, Booking_ID, Audit_ID
FOREIGN KEY	Flight, Schedule, Seat, and Booking reference their parent tables (Table 3)
NOT NULL	Airport_Name, City, Model, Flight_Number, Flight_Date, Seat_Number, Passenger_Name
UNIQUE	Flight_Number, Email, Phone, (Aircraft_ID, Seat_Number)
CHECK	Capacity > 0, Base_Fare > 0, Fare > 0, Available_Seats ≥ 0, Seat_Class and Status in a fixed list, Source_Airport $\neq$ Dest_Airport
Unique index	Uq_Active_Seat: one *confirmed* booking per seat per schedule

Table 6: Summary of constraints

Primary keys uniquely identify records and foreign keys establish relationships. NOT NULL ensures that required fields contain values, UNIQUE prevents duplicates, and CHECK restricts values according to specified conditions. The function-based unique index `Uq_Active_Seat` ignores cancelled bookings (the indexed expressions are NULL for them), so a cancelled seat can be sold again while two confirmed bookings for the same seat and schedule are impossible.

6.3 ALTER Operation

The ALTER command modifies an existing table: it can add a column, change a column definition, or add a constraint.

```
ALTER TABLE Passenger ADD Date_Of_Birth DATE;
ALTER TABLE Aircraft  MODIFY Model VARCHAR2(80);
ALTER TABLE Flight    ADD CONSTRAINT Chk_Flight_Times
      CHECK (Dep_Time <> Arr_Time);
```

6.4 DROP Operation

The DROP command removes a table and its stored records. It is demonstrated on a temporary copy so that the real data is not lost.

```
CREATE TABLE Flight_Archive AS SELECT * FROM Flight;
DROP TABLE Flight_Archive;
```

7 SQL DML Commands

Data Manipulation Language (DML) commands insert, modify, and delete records. In the Airline Reservation System they manage airport, flight, passenger, and booking information. The complete script is `sql/02_dml.sql`.

7.1 INSERT

The INSERT command adds new records to a table. The figure shows representative INSERT commands and their output.

```
SQL> INSERT INTO Airport VALUES (101, 'Rajiv Gandhi International', 'Hyderabad');
1 row created.
SQL> INSERT INTO Aircraft VALUES (11, 'Airbus A320', 180);
1 row created.
SQL> INSERT INTO Flight VALUES (201, 'AI101', 11, 101, 102, '06:00', '08:10', 5500);
1 row created.
SQL> INSERT INTO Schedule VALUES (301, 201, TO_DATE('2026-11-15','YYYY-MM-DD'), 150);
1 row created.
SQL> INSERT INTO Seat VALUES (501, 11, '1A', 'Business');
1 row created.
SQL> INSERT INTO Passenger VALUES (601, 'Ananya', 'ananya@gmail.com', '9876500001');
1 row created.
SQL> INSERT INTO Booking VALUES (701, 601, 301, 501, TO_DATE('2026-10-20','YYYY-MM-DD')
    , 9500, 'Confirmed');
1 row created.
SQL> COMMIT;
Commit complete.
```

Figure 3: INSERT Commands and Output

The sample data set contains 4 airports, 2 aircraft, 3 flights, 3 schedules, 5 seats, 3 passengers, and 3 bookings.

7.2 UPDATE

The UPDATE command modifies existing records. For example, the status of a booking is changed when a passenger cancels a ticket, and a contact number can be corrected.

```
UPDATE Booking   SET Status = 'Cancelled' WHERE Booking_ID = 703;
UPDATE Passenger SET Phone = '9876500039' WHERE Passenger_ID = 603;
COMMIT;
```

7.3 DELETE

The DELETE command removes records. A passenger record that is not referenced by any booking can be deleted, whereas deleting a passenger who has bookings is rejected by the foreign key.

```
INSERT INTO Passenger VALUES (604, 'Kiran', 'kiran@gmail.com', '9876500004');
DELETE FROM Passenger WHERE Passenger_ID = 604;
COMMIT;
```

```
SQL> DELETE FROM Passenger WHERE Passenger_ID = 604;
1 row deleted.

SQL> DELETE FROM Passenger WHERE Passenger_ID = 601;
ERROR at line 1:
ORA-02292: integrity constraint violated - child record found
```

Figure 4: DELETE of an Unreferenced and a Referenced Passenger

DML commands help maintain and update the data stored in the database according to the operations performed by passengers and administrators.

8 SQL Queries

SQL queries retrieve and analyze the data stored in the Airline Reservation System. The system uses SELECT statements, joins, aggregate functions, subqueries, and views to obtain useful information.

8.1 Basic SELECT Queries

The SELECT command retrieves records from a table, with WHERE and ORDER BY used to filter and sort them.

```
SQL> SELECT Flight_ID, Flight_Number, Dep_Time, Arr_Time, Base_Fare FROM Flight;

 FLIGHT_ID FLIGHT_NUM DEP_TIME   ARR_TIME    BASE_FARE
---------- ---------- ---------- ---------- ----------
       201 AI101      06:00      08:10            5500
       202 6E202      09:30      11:00            4200
       203 UK303      14:00      16:45            6100

SQL> SELECT Airport_ID, City FROM Airport ORDER BY City;

AIRPORT_ID CITY
---------- --------------------
       104 Bengaluru
       102 Delhi
       101 Hyderabad
       103 Mumbai
```

Figure 5: Output of Basic SELECT Queries

8.2 Joins

Joins retrieve related information from two or more tables. The first query joins Flight, Schedule, and Airport (twice) to show each dated flight with its route; the second joins Passenger and Booking; the third is a LEFT OUTER JOIN that also lists flights without any booking.

```
SQL> SELECT F.Flight_Number, S.Flight_Date, A1.City AS Source, A2.City AS Destination
  2  FROM Flight F JOIN Schedule S ON F.Flight_ID = S.Flight_ID
  3  JOIN Airport A1 ON F.Source_Airport = A1.Airport_ID
  4  JOIN Airport A2 ON F.Dest_Airport   = A2.Airport_ID;

FLIGHT_NUMBER FLIGHT_DATE SOURCE          DESTINATION
------------- ----------- --------------- ---------------
AI101         15-NOV-26   Hyderabad       Delhi
6E202         16-NOV-26   Hyderabad       Mumbai
UK303         17-NOV-26   Delhi           Bengaluru

SQL> SELECT P.Passenger_Name, B.Booking_ID, B.Status
  2  FROM Passenger P JOIN Booking B ON P.Passenger_ID = B.Passenger_ID;

PASSENGER_NAME       BOOKING_ID STATUS
-------------------- ---------- ----------
Ananya                      701 Confirmed
Rohit                       702 Confirmed
Meera                       703 Cancelled

SQL> SELECT F.Flight_Number, COUNT(B.Booking_ID) AS Bookings
  2  FROM Flight F LEFT JOIN Schedule S ON F.Flight_ID = S.Flight_ID
  3  LEFT JOIN Booking B ON S.Schedule_ID = B.Schedule_ID
  4  GROUP BY F.Flight_Number ORDER BY F.Flight_Number;

FLIGHT_NUMBER  BOOKINGS
------------- ---------
6E202                 1
AI101                 2
UK303                 0
```

Figure 6: Output of Join Queries

8.3 Aggregate Functions

Aggregate functions such as COUNT, SUM, AVG, MAX, and MIN perform calculations on records.

```
SQL> SELECT COUNT(*) AS Total_Bookings FROM Booking;

TOTAL_BOOKINGS
--------------
             3

SQL> SELECT SUM(Fare) AS Confirmed_Revenue FROM Booking WHERE Status = 'Confirmed';

CONFIRMED_REVENUE
-----------------
            15000

SQL> SELECT ROUND(AVG(Base_Fare),2) AS Avg_Fare, MAX(Base_Fare) AS Highest,
  2         MIN(Base_Fare) AS Lowest FROM Flight;

  AVG_FARE    HIGHEST     LOWEST
---------- ---------- ----------
   5266.67       6100       4200
```

Figure 7: Output of Aggregate Functions

8.4 GROUP BY and HAVING

GROUP BY groups records, while HAVING applies conditions to the groups.

```
SQL> SELECT Status, COUNT(*) AS Total FROM Booking GROUP BY Status ORDER BY Status;

STATUS                    TOTAL
-------------------- ----------
Cancelled                     1
Confirmed                     2

SQL> SELECT Status, COUNT(*) AS Total FROM Booking
  2  GROUP BY Status HAVING COUNT(*) > 1;

STATUS                    TOTAL
-------------------- ----------
Confirmed                     2

SQL> SELECT Schedule_ID, SUM(Fare) AS Revenue FROM Booking
  2  WHERE Status = 'Confirmed' GROUP BY Schedule_ID HAVING SUM(Fare) > 10000;

SCHEDULE_ID    REVENUE
----------- ----------
        301      15000
```

Figure 8: Output of GROUP BY and HAVING Queries

8.5 Subqueries

A subquery is a query written inside another query. The first query lists flights whose base fare is above the average; the second is a nested query (four levels) that finds passengers with a confirmed booking on a flight departing from Hyderabad; the third is a correlated subquery using EXISTS.

```
SQL> SELECT Flight_Number, Base_Fare FROM Flight
  2  WHERE Base_Fare > (SELECT AVG(Base_Fare) FROM Flight);

FLIGHT_NUMBER   BASE_FARE
------------- -----------
AI101                5500
UK303                6100

SQL> SELECT Passenger_Name FROM Passenger WHERE Passenger_ID IN
  2   (SELECT Passenger_ID FROM Booking WHERE Status = 'Confirmed' AND Schedule_ID IN
  3     (SELECT Schedule_ID FROM Schedule WHERE Flight_ID IN
  4       (SELECT Flight_ID FROM Flight WHERE Source_Airport =
  5         (SELECT Airport_ID FROM Airport WHERE City = 'Hyderabad'))));

PASSENGER_NAME
--------------------
Ananya
Rohit

SQL> SELECT Passenger_Name FROM Passenger P WHERE EXISTS
  2   (SELECT 1 FROM Booking B WHERE B.Passenger_ID = P.Passenger_ID
  3      AND B.Status = 'Cancelled');

PASSENGER_NAME
--------------------
Meera
```

Figure 9: Output of Subqueries

8.6 Views

A view is a virtual table created from the result of an SQL query. The view `Booking_Details` hides the five-table join and is reused by the reports in Section 13.

```
SQL> CREATE VIEW Booking_Details AS
  2  SELECT B.Booking_ID, P.Passenger_Name, F.Flight_Number, S.Flight_Date,
  3         St.Seat_Number, B.Fare, B.Status
  4  FROM Booking B JOIN Passenger P ON B.Passenger_ID = P.Passenger_ID
  5  JOIN Schedule S ON B.Schedule_ID = S.Schedule_ID
  6  JOIN Flight F   ON S.Flight_ID   = F.Flight_ID
  7  JOIN Seat St    ON B.Seat_ID     = St.Seat_ID;

View created.

SQL> SELECT * FROM Booking_Details;

BOOKING_ID PASSENGER_NAME  FLIGHT_NUMBER FLIGHT_DATE SEAT_NUMBER  FARE STATUS
---------- --------------- ------------- ----------- ----------- ----- ----------
       701 Ananya          AI101         15-NOV-26   1A           9500 Confirmed
       702 Rohit           AI101         15-NOV-26   12C          5500 Confirmed
       703 Meera           6E202         16-NOV-26   5A           4200 Cancelled
```

Figure 10: Creation and Output of Booking Details View

9 Transactions and Concurrency

Transactions are a sequence of database operations performed as a single logical unit. In the Airline Reservation System, transactions ensure that booking and cancellation operations are completed correctly and consistently.

9.1 Transaction Management

A transaction begins with the first DML statement and ends when the changes are either permanently saved or cancelled. Booking a ticket involves checking the seat, creating the booking, and reducing the available seat count; either all of these steps happen or none of them do (atomicity).

9.2 COMMIT, ROLLBACK and SAVEPOINT

COMMIT permanently saves the changes made by a transaction. ROLLBACK cancels the changes made since the beginning of the transaction or since the last SAVEPOINT. SAVEPOINT creates a point within a transaction to which the transaction can be partially rolled back. The script is `sql/09_transactions.sql`.

```
SQL> INSERT INTO Booking VALUES (704, 601, 302, 505, TO_DATE('2026-10-25','YYYY-MM-DD')
    , 4200, 'Confirmed');
1 row created.

SQL> SAVEPOINT booking_save;
Savepoint created.

SQL> UPDATE Booking SET Status = 'Cancelled' WHERE Booking_ID = 704;
1 row updated.

SQL> ROLLBACK TO booking_save;
Rollback complete.

SQL> SELECT Booking_ID, Status FROM Booking WHERE Booking_ID = 704;

BOOKING_ID STATUS
---------- ----------
       704 Confirmed

SQL> ROLLBACK;
Rollback complete.

SQL> SELECT COUNT(*) FROM Booking WHERE Booking_ID = 704;

  COUNT(*)
----------
         0

SQL> UPDATE Passenger SET Email = 'ananya.s@gmail.com' WHERE Passenger_ID = 601;
1 row updated.

SQL> COMMIT;
Commit complete.

SQL> ROLLBACK;
Rollback complete.

SQL> SELECT Email FROM Passenger WHERE Passenger_ID = 601;

EMAIL
--------------------
ananya.s@gmail.com
```

Figure 11: Output of COMMIT, ROLLBACK and SAVEPOINT Operations

The first block shows that ROLLBACK TO `booking_save` undoes only the cancellation, keeping the INSERT, while the full ROLLBACK removes the booking completely. The second block shows that once a change is COMMITted it is permanent, so the later ROLLBACK has no effect.

9.3 Concurrency Control

Concurrency control manages simultaneous transactions that access the same data. In an airline system many passengers try to book the last seats of a popular flight at the same moment.

The problem. Suppose two passengers, in Session A and Session B, both want seat 12D on flight AI101 (Schedule 301). Each session first checks that the seat has no confirmed booking and then inserts a booking. Without protection both checks return 0 before either insert happens, so *both* inserts succeed and the seat is sold twice (a lost-update / double-booking anomaly). The same interleaving applied to `Available_Seats` (both read 150 and both write 149) would lose one update.

The solution. A session locks the schedule row with `SELECT ... FOR UPDATE` before it checks and books. A second session asking for the same lock must wait until the first one ends with COMMIT or ROLLBACK; afterwards its check sees the committed booking and the request is rejected. The `Book_Ticket` procedure (Section 10.2) uses the same technique, and the unique index `Uq_Active_Seat` acts as a final safety net. Seat counts are changed with `Available_Seats = Available_Seats - 1`, which is atomic under the row lock.

Time	**Session A**	**Session B**
T1	Locks Schedule 301 (`FOR UPDATE`)	
T2		Requests the same lock — *waits*
T3	Checks seat 503: 0 confirmed bookings; inserts Booking 704	(still waiting)
T4	`COMMIT` — lock released	
T5		Lock granted; check finds 1 confirmed booking
T6		Booking rejected, `ROLLBACK`

Table 7: Timeline of two concurrent booking transactions

```
-- ---------------------------- SESSION A ----------------------------
SQL> SELECT Schedule_ID, Available_Seats FROM Schedule WHERE Schedule_ID = 301 FOR
    UPDATE;

SCHEDULE_ID AVAILABLE_SEATS
----------- ---------------
        301             150

SQL> SELECT COUNT(*) FROM Booking
  2  WHERE Schedule_ID = 301 AND Seat_ID = 503 AND Status = 'Confirmed';

  COUNT(*)
----------
         0

SQL> INSERT INTO Booking VALUES (704, 601, 301, 503, SYSDATE, 5500, 'Confirmed');
1 row created.

SQL> COMMIT;
Commit complete.

-- ---------------------------- SESSION B ----------------------------
SQL> SELECT Schedule_ID, Available_Seats FROM Schedule WHERE Schedule_ID = 301 FOR
    UPDATE;
        (session waits until Session A commits ...)

SCHEDULE_ID AVAILABLE_SEATS
----------- ---------------
        301             149

SQL> SELECT COUNT(*) FROM Booking
  2  WHERE Schedule_ID = 301 AND Seat_ID = 503 AND Status = 'Confirmed';

  COUNT(*)
----------
         1

Seat 12D is already booked - request rejected.
SQL> ROLLBACK;
Rollback complete.
```

Figure 12: Concurrent Booking Scenario: Locking Prevents Double Booking

If a session skips the lock and inserts anyway, the database still protects the data:

```
SQL> INSERT INTO Booking VALUES (705, 602, 301, 503, SYSDATE, 5500, 'Confirmed');
ERROR at line 1:
ORA-00001: unique constraint (UQ_ACTIVE_SEAT) violated
```

Figure 13: Unique Index as Safety Net Against Duplicate Seat Reservation

Test data from this demonstration is removed afterwards with `DELETE FROM Booking WHERE Booking_ID = 704; COMMIT;`. Thus, transaction management and concurrency control ensure reliable and consistent ticket reservations.

10 PL/SQL Implementation

PL/SQL is a procedural extension of SQL that allows programming constructs such as variables, conditions, loops, procedures, functions, cursors, and triggers. In the Airline Reservation System, PL/SQL implements the business operations and automates repetitive tasks.

10.1 Functions

A function is a stored PL/SQL program that accepts parameters and returns a value. Two functions are implemented: `Get_Booking_Count` returns the number of bookings of a passenger, and `Calculate_Refund` returns the refund for a cancelled booking (80% of the fare, 0 otherwise, and -1 for an invalid booking).

```
CREATE OR REPLACE FUNCTION Get_Booking_Count(P_ID NUMBER)
RETURN NUMBER IS
    Total_Bookings NUMBER;
BEGIN
    SELECT COUNT(*) INTO Total_Bookings
    FROM Booking
    WHERE Passenger_ID = P_ID;
    RETURN Total_Bookings;
END;
/

CREATE OR REPLACE FUNCTION Calculate_Refund(B_ID NUMBER)
RETURN NUMBER IS
    V_Fare   Booking.Fare%TYPE;
    V_Status Booking.Status%TYPE;
BEGIN
    SELECT Fare, Status INTO V_Fare, V_Status
    FROM Booking
    WHERE Booking_ID = B_ID;

    IF V_Status = 'Cancelled' THEN
        RETURN ROUND(V_Fare * 0.80, 2);    -- 80% refund policy
    ELSE
        RETURN 0;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN -1;                         -- invalid booking id
END;
/
```

```
SQL> SELECT Get_Booking_Count(601) AS Bookings FROM DUAL;

  BOOKINGS
----------
         1

SQL> SELECT Calculate_Refund(703) AS Refund FROM DUAL;

    REFUND
----------
      3360

SQL> SELECT Calculate_Refund(999) AS Refund FROM DUAL;

    REFUND
----------
        -1
```

Figure 14: Output of PL/SQL Functions

10.2 Stored Procedures

A stored procedure is a named PL/SQL program that performs a major business operation. `Book_Ticket` validates the request, locks the schedule, rejects unavailable or invalid seats, and creates the booking; `Cancel_Booking` cancels a booking and reports the refund.

```
CREATE OR REPLACE PROCEDURE Book_Ticket(
    P_Passenger NUMBER,
    P_Schedule  NUMBER,
    P_Seat      NUMBER,
    P_Fare      NUMBER) AS
    V_Avail NUMBER;
    V_Taken NUMBER;
    V_Valid NUMBER;
    V_Next  NUMBER;
BEGIN
    -- lock the schedule row so concurrent bookings are serialised
    SELECT Available_Seats INTO V_Avail
    FROM Schedule WHERE Schedule_ID = P_Schedule FOR UPDATE;

    SELECT COUNT(*) INTO V_Valid
    FROM Seat St JOIN Flight F ON F.Aircraft_ID = St.Aircraft_ID
                 JOIN Schedule S ON S.Flight_ID = F.Flight_ID
    WHERE St.Seat_ID = P_Seat AND S.Schedule_ID = P_Schedule;

    SELECT COUNT(*) INTO V_Taken FROM Booking
    WHERE Schedule_ID = P_Schedule AND Seat_ID = P_Seat
      AND Status = 'Confirmed';

    IF V_Avail <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'No seats available');
    ELSIF V_Valid = 0 THEN
        RAISE_APPLICATION_ERROR(-20004, 'Seat does not belong to this flight');
    ELSIF V_Taken > 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Seat already booked');
    END IF;

    SELECT NVL(MAX(Booking_ID), 700) + 1 INTO V_Next FROM Booking;
    INSERT INTO Booking
    VALUES (V_Next, P_Passenger, P_Schedule, P_Seat,
            SYSDATE, P_Fare, 'Confirmed');
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Booking confirmed. Booking ID: ' || V_Next);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20003, 'Invalid schedule');
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
/

CREATE OR REPLACE PROCEDURE Cancel_Booking(B_ID NUMBER) AS
    V_Status Booking.Status%TYPE;
BEGIN
    SELECT Status INTO V_Status
```

```
    FROM Booking WHERE Booking_ID = B_ID FOR UPDATE;

    IF V_Status = 'Cancelled' THEN
        RAISE_APPLICATION_ERROR(-20005, 'Booking already cancelled');
    END IF;

    UPDATE Booking SET Status = 'Cancelled'
    WHERE Booking_ID = B_ID;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Booking cancelled. Refund: Rs. '
                         || Calculate_Refund(B_ID));
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20006, 'Booking not found');
END;
/
```

```
SQL> SET SERVEROUTPUT ON
SQL> EXEC Book_Ticket(602, 303, 503, 6100);
Booking confirmed. Booking ID: 704

SQL> EXEC Book_Ticket(602, 301, 502, 5500);
ERROR at line 1:
ORA-20002: Seat already booked

SQL> EXEC Book_Ticket(602, 302, 503, 4200);
ERROR at line 1:
ORA-20004: Seat does not belong to this flight

SQL> EXEC Cancel_Booking(704);
Booking cancelled. Refund: Rs. 4880

SQL> EXEC Cancel_Booking(704);
ERROR at line 1:
ORA-20005: Booking already cancelled

SQL> DELETE FROM Booking WHERE Booking_ID = 704;   -- remove test data
SQL> COMMIT;
```

Figure 15: Execution of Stored Procedures

10.3 Cursors

A cursor processes the rows returned by a query one at a time. An *explicit cursor* prepares the revenue report of every flight, and the *implicit cursor* attribute `SQL%ROWCOUNT` reports how many bookings a batch cancellation (for example when a flight is called off) affected.

```
-- Explicit cursor: revenue report for every flight
SET SERVEROUTPUT ON
DECLARE
    CURSOR Flight_Cursor IS
        SELECT F.Flight_Number,
               NVL(SUM(CASE WHEN B.Status = 'Confirmed'
                            THEN B.Fare END), 0) AS Revenue
        FROM Flight F
        LEFT JOIN Schedule S ON S.Flight_ID = F.Flight_ID
        LEFT JOIN Booking  B ON B.Schedule_ID = S.Schedule_ID
        GROUP BY F.Flight_Number
        ORDER BY F.Flight_Number;
BEGIN
```

```
    FOR R IN Flight_Cursor LOOP
        DBMS_OUTPUT.PUT_LINE(RPAD(R.Flight_Number, 10) ||
                             'Rs. ' || R.Revenue);
    END LOOP;
END;
/

-- Implicit cursor: batch cancellation when a flight is called off
CREATE OR REPLACE PROCEDURE Cancel_Schedule(P_Schedule NUMBER) AS
BEGIN
    UPDATE Booking SET Status = 'Cancelled'
    WHERE Schedule_ID = P_Schedule AND Status = 'Confirmed';
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' booking(s) cancelled.');
    COMMIT;
END;
/
```

```
6E202      Rs. 0
AI101      Rs. 15000
UK303      Rs. 0

PL/SQL procedure successfully completed.

SQL> EXEC Cancel_Schedule(303);
0 booking(s) cancelled.
```

Figure 16: Output of the Explicit and Implicit Cursors

10.4 Triggers

A trigger is a PL/SQL program that executes automatically when a specified database event occurs. Three triggers are defined on Booking: `Booking_Validate` sets defaults and rejects bookings made after the flight date, `Seat_Inventory` keeps `Schedule.Available_Seats` up to date, and `Booking_Audit_Trg` records the history of every change in `Booking_Audit`.

```
-- 1. Validation and default values
CREATE OR REPLACE TRIGGER Booking_Validate
BEFORE INSERT ON Booking
FOR EACH ROW
DECLARE
    V_Date DATE;
BEGIN
    IF :NEW.Status IS NULL THEN
        :NEW.Status := 'Confirmed';
    END IF;
    IF :NEW.Booking_Date IS NULL THEN
        :NEW.Booking_Date := SYSDATE;
    END IF;
    SELECT Flight_Date INTO V_Date
    FROM Schedule WHERE Schedule_ID = :NEW.Schedule_ID;
    IF TRUNC(:NEW.Booking_Date) > V_Date THEN
        RAISE_APPLICATION_ERROR(-20010,
            'Cannot book a flight that has already departed');
    END IF;
END;
```

```
/

-- 2. Automatic seat inventory update
CREATE OR REPLACE TRIGGER Seat_Inventory
AFTER INSERT OR UPDATE OF Status OR DELETE ON Booking
FOR EACH ROW
BEGIN
    IF INSERTING AND :NEW.Status = 'Confirmed' THEN
        UPDATE Schedule SET Available_Seats = Available_Seats - 1
        WHERE Schedule_ID = :NEW.Schedule_ID;
    ELSIF UPDATING AND :OLD.Status = 'Confirmed'
                   AND :NEW.Status = 'Cancelled' THEN
        UPDATE Schedule SET Available_Seats = Available_Seats + 1
        WHERE Schedule_ID = :NEW.Schedule_ID;
    ELSIF DELETING AND :OLD.Status = 'Confirmed' THEN
        UPDATE Schedule SET Available_Seats = Available_Seats + 1
        WHERE Schedule_ID = :OLD.Schedule_ID;
    END IF;
END;
/

-- 3. Audit / history logging
CREATE OR REPLACE TRIGGER Booking_Audit_Trg
AFTER INSERT OR UPDATE OR DELETE ON Booking
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO Booking_Audit (Booking_ID, Action_Type, Old_Status, New_Status)
        VALUES (:NEW.Booking_ID, 'INSERT', NULL, :NEW.Status);
    ELSIF UPDATING THEN
        INSERT INTO Booking_Audit (Booking_ID, Action_Type, Old_Status, New_Status)
        VALUES (:NEW.Booking_ID, 'UPDATE', :OLD.Status, :NEW.Status);
    ELSE
        INSERT INTO Booking_Audit (Booking_ID, Action_Type, Old_Status, New_Status)
        VALUES (:OLD.Booking_ID, 'DELETE', :OLD.Status, NULL);
    END IF;
END;
/
```

```
SQL> INSERT INTO Booking (Booking_ID, Passenger_ID, Schedule_ID, Seat_ID, Fare)
  2  VALUES (705, 601, 302, 505, 4200);
1 row created.

SQL> SELECT Booking_ID, Status FROM Booking WHERE Booking_ID = 705;

BOOKING_ID STATUS
---------- ----------
       705 Confirmed

SQL> SELECT Schedule_ID, Available_Seats FROM Schedule WHERE Schedule_ID = 302;

SCHEDULE_ID AVAILABLE_SEATS
----------- ---------------
        302             119

SQL> SELECT Booking_ID, Action_Type, Old_Status, New_Status
  2  FROM Booking_Audit WHERE Booking_ID = 705;

BOOKING_ID ACTION_TYP OLD_STATUS NEW_STATUS
---------- ---------- ---------- ----------
       705 INSERT                Confirmed

SQL> INSERT INTO Booking (Booking_ID, Passenger_ID, Schedule_ID, Seat_ID, Booking_Date,
     Fare)
  2  VALUES (706, 602, 302, 504, TO_DATE('2026-11-20','YYYY-MM-DD'), 4200);
ERROR at line 1:
ORA-20010: Cannot book a flight that has already departed

SQL> ROLLBACK;
Rollback complete.
```

Figure 17: Output of Trigger Execution

The first insert omits the status and date, which the trigger fills in; the seat count falls from 120 to 119 and an audit row is written. The second insert is rejected by the validation trigger. The final ROLLBACK undoes the test data. PL/SQL functions, procedures, cursors, and triggers improve automation, reduce repeated operations, and help maintain consistency in the database.

11 File Organization and Indexing

File organization defines how records are stored and arranged in a database file. Indexing improves the speed of data retrieval by providing an efficient way to locate records. In the Airline Reservation System the most frequently searched attributes are Flight_ID, Schedule_ID, Seat_ID, Passenger_ID, and Booking_ID.

11.1 File Organization

Organization	How records are stored	Use in this project
Sequential	Records kept in order of a key; good for full scans and reports	Flight records ordered by Flight_ID for the daily flight listing
Heap	Records appended wherever space is free; fast inserts, slow searches	Default Oracle table storage; used for Booking, where new rows arrive continuously
Indexed	Records located through an index on a key	Index-organized table and B+ tree indexes for Booking_ID, Passenger_ID, and Flight_ID lookups

Table 8: File organizations considered

Oracle stores ordinary tables as heaps, so sequential retrieval is obtained with ORDER BY. Indexed file organization is demonstrated with an index-organized table, whose rows are stored inside the primary-key B+ tree itself:

```
CREATE TABLE Booking_IOT (
    Booking_ID  INT PRIMARY KEY,
    Schedule_ID INT,
    Fare        NUMBER(10,2)
) ORGANIZATION INDEX;
```

11.2 ISAM

Indexed Sequential Access Method (ISAM) stores records sequentially in primary data pages and keeps a *static* index over them. It supports both sequential and direct access. The index is built once; when a data page becomes full, new records are chained to an *overflow page*, so the structure degrades if many records are inserted. Figure 18 shows an ISAM file on Seat_ID (page capacity two keys) after Seat_ID 507 is inserted into the full page holding 505 and 506. A search for Seat_ID 507 reads the index page, follows the pointer to the third data page, and then follows its overflow chain.

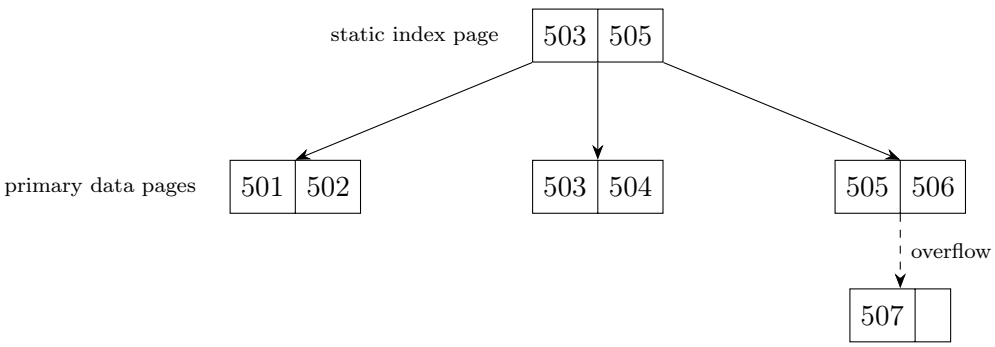


Figure 18: ISAM Structure on Seat_ID with an Overflow Page

11.3 B+ Tree Indexing

A B+ tree is a balanced tree structure commonly used for database indexing. Internal nodes hold only index keys, the leaf nodes hold the keys with record pointers, and the leaves are linked, which makes sequential and range searches efficient. Unlike ISAM, a B+ tree splits nodes as it grows and therefore stays balanced without overflow chains. Oracle's default index is a B+ tree. Figure 19 shows a B+ tree on Seat_ID.

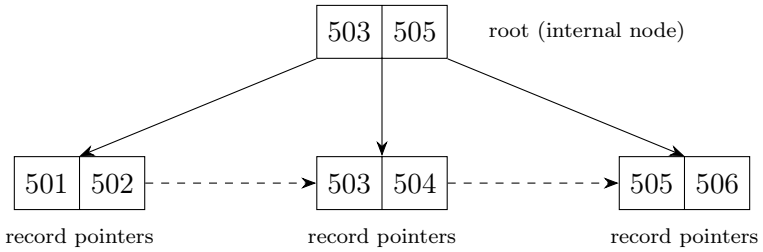


Figure 19: B+ Tree Index on Seat_ID (dashed arrows: linked leaf nodes)

The primary keys are indexed automatically. The following secondary indexes speed up the most common searches (script `sql/08_indexes.sql`):

```
CREATE INDEX Idx_Booking_Passenger ON Booking(Passenger_ID);
CREATE INDEX Idx_Schedule_Flight   ON Schedule(Flight_ID);
CREATE INDEX Idx_Flight_Route      ON Flight(Source_Airport, Dest_Airport);
```

The effect is visible in the execution plan. On a table with only a few rows the optimizer may still choose a full table scan; with a large number of bookings the index is used, as shown below.

```
SQL> EXPLAIN PLAN FOR SELECT * FROM Booking WHERE Passenger_ID = 602;
Explained.

SQL> SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

-------------------------------------------------------------------------------------
| Id | Operation                           | Name                 | Rows | Cost |
-------------------------------------------------------------------------------------
|  0 | SELECT STATEMENT                    |                      |    1 |    2 |
|  1 |  TABLE ACCESS BY INDEX ROWID BATCHED| BOOKING              |    1 |    2 |
|* 2 |   INDEX RANGE SCAN                  | IDX_BOOKING_PASSENGER|    1 |    1 |
-------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
   2 - access("PASSENGER_ID"=602)
```

Figure 20: Execution Plan Using the B+ Tree Index

B+ tree indexing improves search performance and reduces the number of database blocks that must be accessed. It is especially useful when the database contains a large number of flight, schedule, and booking records.

12 Testing

Testing verifies that the Airline Reservation System works correctly and produces the expected results. Test cases cover normal operations, boundary conditions, invalid inputs, and transaction and concurrency behaviour.

12.1 Test Cases

The following test cases verify the main functions of the system.

Test ID	Test case	Expected result	Status
TC01	Search flights Hyderabad to Delhi	Flight AI101 on 15-NOV-26 is displayed	Pass
TC02	Book an available seat with `Book_Ticket`	Booking created with status Confirmed; Available_Seats reduced by 1	Pass
TC03	Cancel a valid booking	Status becomes Cancelled; seat count restored; refund = 80% of fare	Pass
TC04	View all bookings through `Booking_Details`	All bookings with passenger and flight names are listed	Pass
TC05	Run the revenue report	Revenue equals the sum of confirmed fares (15000 for AI101)	Pass

Table 9: Functional test cases

12.2 Normal and Boundary Cases

Normal cases use valid inputs, such as selecting an available seat and entering valid passenger details. Boundary cases test values at the limits.

Test ID	Boundary case	Expected result	Status
TC06	Book the last seat (Available_Seats = 1)	Booking succeeds; Available_Seats becomes 0	Pass
TC07	Book when Available_Seats = 0	Rejected with ORA-20001 (No seats available)	Pass
TC08	Insert aircraft with Capacity = 1 and Capacity = 0	1 accepted; 0 rejected by CHECK (ORA-02290)	Pass
TC09	Booking_Date equal to Flight_Date / one day after	Same day accepted; next day rejected (ORA-20010)	Pass
TC10	First seat (1A) and last seat of an aircraft	Both can be booked normally	Pass

Table 10: Normal and boundary test cases

12.3 Invalid Input Testing

Invalid input testing verifies that incorrect data is rejected and database consistency is maintained.

Test ID	Invalid input	Expected result	Status
TC11	Booking for a non-existing passenger ID	ORA-02291 parent key not found	Pass
TC12	Booking Status = 'Pending'	ORA-02290 CHECK constraint violated	Pass
TC13	Passenger with an existing e-mail	ORA-00001 unique constraint violated	Pass
TC14	Flight with Source_Airport = Dest_Airport	ORA-02290 CHECK constraint violated	Pass
TC15	Negative or zero fare	ORA-02290 CHECK constraint violated	Pass
TC16	Delete a passenger who has bookings	ORA-02292 child record found	Pass
TC17	Cancel an already cancelled booking	ORA-20005 Booking already cancelled	Pass
TC18	Book a seat of another aircraft	ORA-20004 Seat does not belong to this flight	Pass

Table 11: Invalid input test cases

12.4 Transaction and Concurrency Testing

Test ID	Scenario	Expected result	Status
TC19	Insert a booking, then ROLLBACK	Booking does not exist afterwards	Pass
TC20	Change a booking, SAVEPOINT, change again, ROLLBACK TO savepoint	Only the second change is undone	Pass
TC21	COMMIT followed by ROLLBACK	Committed change remains	Pass
TC22	Two sessions book the same seat at the same time	Second session waits, then is rejected; one confirmed booking only	Pass
TC23	Two sessions book different seats on the same flight	Both succeed; Available_Seats reduced by exactly 2	Pass
TC24	Booking inserted without locking while seat is taken	ORA-00001 from Uq_Active_Seat	Pass

Table 12: Transaction and concurrency test cases

Testing ensures that the system performs its operations correctly and helps identify errors before the system is used in practice.

13 Reports

The Airline Reservation System generates reports that provide useful information about ticket bookings, cancellations, and revenue. These reports help administrators monitor the activities of the system and make better decisions. All reports are produced with SQL queries over the data set of Section 7.

13.1 Booking Report

The booking report lists the tickets booked by passengers with flight, seat, fare, and status.

```
SELECT Booking_ID, Passenger_Name, Flight_Number, Seat_Number, Fare, Status
FROM Booking_Details ORDER BY Booking_ID;
```

Booking ID	Passenger	Flight	Seat	Fare	Status
701	Ananya	AI101	1A	Rs. 9500	Confirmed
702	Rohit	AI101	12C	Rs. 5500	Confirmed
703	Meera	6E202	5A	Rs. 4200	Cancelled

Table 13: Sample Booking Report

13.2 Cancellation Report

The cancellation report displays the cancelled bookings and the refund payable, helping administrators track cancelled tickets.

```
SELECT B.Booking_ID, P.Passenger_Name, B.Booking_Date, B.Fare,
       Calculate_Refund(B.Booking_ID) AS Refund
FROM Booking B JOIN Passenger P ON B.Passenger_ID = P.Passenger_ID
WHERE B.Status = 'Cancelled';
```

Booking ID	Passenger	Booking Date	Fare	Refund
703	Meera	2026-10-22	Rs. 4200	Rs. 3360

Table 14: Sample Cancellation Report

13.3 Revenue Report

The revenue report gives the income generated from confirmed bookings for every dated flight, so that administrators can analyze the financial performance of each route.

```
SELECT F.Flight_Number, S.Flight_Date,
       COUNT(CASE WHEN B.Status = 'Confirmed' THEN 1 END) AS Tickets,
       NVL(SUM(CASE WHEN B.Status = 'Confirmed' THEN B.Fare END), 0) AS Revenue
FROM Flight F JOIN Schedule S ON S.Flight_ID = F.Flight_ID
LEFT JOIN Booking B ON B.Schedule_ID = S.Schedule_ID
```

```
GROUP BY F.Flight_Number, S.Flight_Date
ORDER BY S.Flight_Date;
```

Flight	Flight Date	Tickets	Revenue
AI101	2026-11-15	2	Rs. 15000
6E202	2026-11-16	0	Rs. 0
UK303	2026-11-17	0	Rs. 0

Table 15: Sample Revenue Report

The same revenue figures are produced by the cursor-based procedure of Section 10.3. These reports provide a clear summary of booking activities, cancellations, and revenue, helping administrators manage the airline reservation system efficiently.

14 Results and Discussion

The Airline Reservation System successfully provides a structured database for managing airports, aircraft, flights, schedules, seats, passengers, and bookings. The implemented SQL commands allow records to be created, updated, deleted, and retrieved efficiently.

The database design reduces data redundancy through normalization up to 3NF and maintains data integrity using primary keys, foreign keys, UNIQUE, NOT NULL, and CHECK constraints. Transactions and locking prevent the same seat from being sold twice when several passengers book at the same time, and the unique index on confirmed bookings guarantees this even if the application code is bypassed.

The implemented PL/SQL functions, procedures, cursors, and triggers provide additional automation: seat counts are updated automatically, every change to a booking is logged, and refunds are calculated by a reusable function. Indexing speeds up searches on the most frequently used columns. The testing results show that the system handles normal operations, boundary values, invalid inputs, seat reservations, cancellations, and concurrent bookings appropriately. The generated booking, cancellation, and revenue reports provide useful information for administrators.

Script	Contents
01_ddl.sql	Table creation, constraints, unique index, audit table
02_dml.sql	INSERT, UPDATE, DELETE statements and sample data
03_alter_drop.sql	ALTER and DROP operations
04_functions.sql	PL/SQL functions
05_procedures.sql	Stored procedures
06_cursors.sql	Explicit and implicit cursors
07_triggers.sql	Validation, inventory, and audit triggers
08_indexes.sql	Index-organized table, B+ tree indexes, execution plan
09_transactions.sql	COMMIT, ROLLBACK, SAVEPOINT demonstration
10_queries.sql	Joins, aggregates, GROUP BY, HAVING, subqueries, views, reports
11_concurrency.sql	Two-session concurrency scenario

Table 16: SQL scripts supplied with the report

Overall, the system meets the required database functionalities and provides a reliable and efficient approach for managing airline ticket reservations.

15 Conclusion

The Airline Reservation System provides an efficient database-based solution for managing airline ticket reservations. The system organizes information related to airports, aircraft, flights, schedules, seats, passengers, and bookings in a structured manner.

The project demonstrates important DBMS concepts including ER modeling, relational schema design, normalization, SQL DDL and DML commands, queries, transactions, concurrency control, PL/SQL functions, procedures, cursors, triggers, file organization, and indexing. These concepts help maintain data integrity, reduce redundancy, and improve the efficiency of database operations.

The testing and reports demonstrate that the system can handle flight searches, seat reservations, cancellations, and booking-related information effectively. Overall, the proposed system provides a reliable and organized approach to airline reservation and can be further enhanced with features such as online payment, notifications, and user authentication.

16 SDG Mapping

The Airline Reservation System contributes to the Sustainable Development Goals (SDGs) by promoting digitalization, efficient information management, and paperless services.

SDG Goal	Contribution of the Project
SDG 9 Industry, Innovation and Infrastructure	The system uses database technology and digital services to improve the efficiency and reliability of airline ticket booking and information management.
SDG 11 Sustainable Cities and Communities	Digital ticket reservation reduces manual processes and supports organized digital services for transportation and travel.
SDG 12 Responsible Consumption and Production	The system reduces paper usage by maintaining digital booking, cancellation, and passenger records.

Table 17: SDG Mapping of the Project

The project demonstrates how database technology can support efficient digital services while reducing manual record keeping and paper-based processes.

17 Future Scope

The Airline Reservation System can be further enhanced with additional features to improve its functionality and user experience. Online payment can be integrated to allow passengers to complete ticket purchases digitally. User authentication can be added to provide secure access to passenger accounts and booking history.

The system can also be extended with real-time seat maps, connecting flights and multi-city search, meal and baggage options, email or SMS notifications, and a mobile application. A fare-prediction or dynamic-pricing module can be introduced based on demand and booking history. These enhancements can make the system more secure, convenient, and suitable for real-world deployment.

18 References

[1] Raghurama Krishnan and Johannes Gehrke, *Database Management Systems*, 3rd ed., Tata McGraw-Hill, 2014.

[2] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill, 2019.

[3] Ramez Elmasri and Shamkant Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson Education, 2016.

[4] C. J. Date, A. Kannan, and S. Swamynathan, *An Introduction to Database Systems*, 8th ed., Pearson Education India, 2015.

[5] Oracle Corporation, *Oracle Database SQL Language Reference* and *PL/SQL Language Reference*, Oracle Documentation.